

# MANUAL DE USUARIO

## Clasificador de Enfermedad Cardíaca mediante Machine Learning

Proyecto 3 - CE en Desarrollo de aplicaciones en lenguaje Python



# Índice

---

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>1. Requisitos previos del sistema.....</b>         | <b>2</b>  |
| Software necesario.....                               | 2         |
| Hardware recomendado.....                             | 3         |
| <b>2. Instalación paso a paso.....</b>                | <b>3</b>  |
| Paso 1 - Clonar el repositorio.....                   | 3         |
| Paso 2 - Crear un entorno virtual.....                | 3         |
| Paso 3 - Instalar dependencias.....                   | 3         |
| Paso 4 - Verificar la instalación.....                | 4         |
| <b>4. Cómo arrancar la aplicación.....</b>            | <b>6</b>  |
| Paso 1 - Entrenar los modelos.....                    | 6         |
| Paso 2 - Evaluar los modelos.....                     | 6         |
| <b>5. Uso de cada funcionalidad.....</b>              | <b>7</b>  |
| Análisis exploratorio - analisis.ipynb.....           | 7         |
| Preprocesamiento - preprocessing.py.....              | 7         |
| Entrenamiento - train_model.py.....                   | 7         |
| Evaluación — evaluation.py.....                       | 7         |
| <b>6. Herramientas externas necesarias.....</b>       | <b>8</b>  |
| Extensiones recomendadas para VS Code.....            | 8         |
| <b>7. Solución a errores comunes.....</b>             | <b>9</b>  |
| Error: ModuleNotFoundError.....                       | 9         |
| Error: FileNotFoundError - heart.csv.....             | 9         |
| Error: ValueError - columnas category en XGBoost..... | 9         |
| Error: models/ no encontrada al evaluar.....          | 9         |
| Warning: FutureWarning en seaborn.....                | 9         |
| <b>8. Mensajes del sistema.....</b>                   | <b>10</b> |
| Ejecución exitosa completa.....                       | 10        |

## 1. Requisitos previos del sistema

Antes de instalar y ejecutar el proyecto, asegúrese de que su sistema cumple los siguientes requisitos:

### Software necesario

| Requisito          | Versión mínima  | Notas                                        |
|--------------------|-----------------|----------------------------------------------|
| Python             | 3.9 o superior  | Verificar con: <code>python --version</code> |
| pip                | 21.0 o superior | Incluido con Python.                         |
| Git                | 2.x             | Para clonar el repositorio                   |
| Visual Studio Code | Cualquiera      | Recomendado. Opcional.                       |

### Hardware recomendado

- RAM: 4 GB mínimo (8 GB recomendado)
- Espacio en disco: 500 MB libres
- Sistema operativo: Windows 10/11, macOS 10.15+, o Ubuntu 20.04+

NOTA: El proyecto no requiere GPU. Todo el procesamiento se realiza en CPU.

## 2. Instalación paso a paso

### Paso 1 - Clonar el repositorio

Abra una terminal y ejecute el siguiente comando:

```
git clone
https://github.com/marquineo/CLASIFICADOR-DE-ESTADO-DE-SALUD-MEDIANTE-
MACHINE-LEARNING.git
cd CLASIFICADOR-DE-ESTADO-DE-SALUD-MEDIANTE-MACHINE-LEARNING
```

### Paso 2 - Crear un entorno virtual

Se recomienda usar un entorno virtual para aislar las dependencias del proyecto:

En Windows:

```
python -m venv .venv
.venv\Scripts\activate
```

En macOS / Linux:

```
python -m venv .venv
source .venv/bin/activate
```

El entorno virtual está activo cuando aparece (.venv) al inicio de la línea de comandos.

### Paso 3 - Instalar dependencias

```
pip install -r requirements.txt
```

Las librerías instaladas son:

| Librería     | Versión        |
|--------------|----------------|
| pandas       | 3.0.2          |
| numpy        | 2.4.1          |
| matplotlib   | 3.10.8         |
| seaborn      | 0.13.2         |
| scikit-learn | 1.8.0          |
| xgboost      | última estable |
| joblib       | 1.5.3          |
| jupyter      | 1.1.1          |

### Paso 4 - Verificar la instalación

Compruebe que todo se ha instalado correctamente:

```
python -c "import pandas, numpy, sklearn, xgboost, seaborn;
print('OK')"
```

Si el comando imprime OK, la instalación es correcta y puede continuar.

### 3. Configuración inicial

---

Este proyecto no requiere variables de entorno ni credenciales externas. No obstante, verifique que la estructura de carpetas sea correcta antes de ejecutar:

```
CLASIFICADOR-DE-ESTADO-DE-SALUD-MEDIANTE-MACHINE-LEARNING/  
├── data/  
│   └── heart.csv          ← el dataset debe estar aquí  
├── models/               ← se crea automáticamente al entrenar  
├── notebooks/  
│   └── analisis.ipynb  
├── src/  
│   ├── preprocessing.py  
│   ├── train_model.py  
│   └── evaluation.py  
├── images/               ← se guardan las gráficas aquí  
├── README.md  
└── requirements.txt
```

**IMPORTANTE:** El archivo heart.csv debe estar en la carpeta data/ antes de ejecutar cualquier script.

## 4. Cómo arrancar la aplicación

---

El proyecto se ejecuta en dos pasos secuenciales desde la carpeta src/:

### Paso 1 - Entrenar los modelos

Este script carga los datos, aplica el preprocesamiento y entrena los tres modelos de clasificación:

```
cd src
python train_model.py
```

Output esperado:

```
Entrenando Logistic Regression...
  CV Accuracy: 0.849 +/- 0.026
Entrenando Random Forest...
  CV Accuracy: 0.846 +/- 0.019
Entrenando XGBoost...
  CV Accuracy: 0.849 +/- 0.010
Guardado: logistic_regression.pkl
Guardado: random_forest.pkl
Guardado: xgboost.pkl
```

### Paso 2 - Evaluar los modelos

Este script carga los modelos entrenados, calcula las métricas sobre el conjunto de test y genera las gráficas:

```
python evaluation.py
```

Output esperado:

```

Accuracy Precision Recall    F1    AUC
Logistic Regression    0.897    0.895    0.922    0.908    0.935
Random Forest          0.908    0.897    0.941    0.919    0.933
Xgboost                0.886    0.901    0.892    0.897    0.944

Logistic Regression - Recall por sexo:
  M: 0.938
  F: 0.667
Random Forest - Recall por sexo:
  M: 0.948
  F: 0.833
```

## 5. Uso de cada funcionalidad

---

### Análisis exploratorio - analisis.ipynb

Para ejecutar el notebook de análisis exploratorio:

```
cd notebooks
jupyter notebook analisis.ipynb
```

El notebook se abrirá en el navegador. Ejecute las celdas en orden con Shift + Enter. Contiene:

- Vista general del dataset (head, shape, info, describe)
- Análisis de tipos de datos y conversión a category
- Detección de valores nulos, duplicados y ceros imposibles
- Distribuciones de variables numéricas y categóricas
- Heatmap de correlaciones
- Boxplots y countplots de variables vs HeartDisease
- Conclusiones del EDA

### Preprocesamiento - preprocessing.py

Este módulo no se ejecuta directamente. Es llamado automáticamente por train\_model.py y evaluation.py. Si desea usarlo de forma independiente:

```
from preprocessing import preprocess

X_train, X_test, y_train, y_test, scaler =
preprocess('../data/heart.csv')
print(X_train.shape) # (734, 18) aprox.
```

### Entrenamiento - train\_model.py

Entrena los tres modelos y los guarda en models/. Los archivos generados son:

- models/logistic\_regression.pkl: modelo de regresión logística
- models/random\_forest.pkl: modelo de random forest (seleccionado como mejor)
- models/xgboost.pkl: modelo de XGBoost
- models/scaler.pkl: escalador StandardScaler entrenado sobre train

### Evaluación — evaluation.py

Evalúa los modelos y genera las siguientes gráficas en images/:

- numericas\_vs\_target.png: boxplots de variables numéricas vs HeartDisease
- categoricas\_vs\_target.png: countplots de variables categóricas vs HeartDisease
- confusion\_matrices.png: matrices de confusión de los tres modelos
- roc\_curves.png: curvas ROC comparativas
- metrics\_comparison.png: gráfico de barras con todas las métricas

## 6. Herramientas externas necesarias

---

Este proyecto es completamente local y de línea de comandos. No requiere herramientas externas como Postman o servicios en la nube. Las únicas herramientas necesarias son:

| Herramienta           | Uso                                 | Cómo acceder                                                                  |
|-----------------------|-------------------------------------|-------------------------------------------------------------------------------|
| Terminal / PowerShell | Ejecutar los scripts .py            | Integrada en el sistema operativo                                             |
| Jupyter Notebook      | Ejecutar el EDA interactivo         | Se lanza con: jupyter notebook                                                |
| Visual Studio Code    | Editar y revisar el código          | Descarga en <a href="https://code.visualstudio.com">code.visualstudio.com</a> |
| GitHub                | Acceder al repositorio del proyecto | <a href="https://github.com">github.com</a>                                   |

### Extensiones recomendadas para VS Code

- Python: soporte completo para Python
- Jupyter: ejecutar notebooks dentro de VS Code
- Pylance: autocompletado y análisis de código



## 7. Solución a errores comunes

---

### Error: ModuleNotFoundError

```
ModuleNotFoundError: No module named 'xgboost'
```

Causa: la librería no está instalada o el entorno virtual no está activo.

Solución:

```
.venv\Scripts\activate    # Windows  
pip install -r requirements.txt
```

### Error: FileNotFoundError - heart.csv

```
FileNotFoundError: [Errno 2] No such file or directory:  
'../data/heart.csv'
```

Causa: el script se ejecuta desde una carpeta incorrecta o el CSV no está en data/.

Solución: asegúrese de ejecutar desde src/ y que el archivo existe en data/:

```
cd src                    # ejecutar siempre desde src/  
ls ../data/              # verificar que heart.csv existe
```

### Error: ValueError - columnas category en XGBoost

```
ValueError: DataFrame.dtypes for data must be int, float, bool or  
category...
```

Causa: alguna columna tiene tipo category y XGBoost no la acepta sin configuración adicional.

Solución: verificar que FastingBS se convierte a int64 en preprocessing.py:

```
df['FastingBS'] = df['FastingBS'].astype('int64')
```

### Error: models/ no encontrada al evaluar

```
FileNotFoundError: [Errno 2] No such file or directory: '../models/'
```

Causa: se ejecutó evaluation.py antes de train\_model.py.

Solución: ejecutar siempre en orden:

1. python train\_model.py
2. python evaluation.py

### Warning: FutureWarning en seaborn

```
FutureWarning: Passing palette without assigning hue is deprecated...
```

Causa: versión de seaborn >= 0.13 cambió la API de palette.

Solución: añadir hue y legend=False en las llamadas a sns.boxplot():

```
sns.boxplot(x='HeartDisease', y=col, hue='HeartDisease', legend=False,  
...)
```

Los warnings no impiden la ejecución. Solo los errores (Error:) detienen el programa.

## 8. Mensajes del sistema

A continuación se describen los principales mensajes que puede ver durante la ejecución:

| Mensaje                           | Tipo    | Significado                                               |
|-----------------------------------|---------|-----------------------------------------------------------|
| CV Accuracy: 0.849 +/- 0.026      | INFO    | Resultado de la validación cruzada. Normal y esperado.    |
| Guardado: random_forest.pkl       | INFO    | Modelo serializado correctamente en models/.              |
| FutureWarning: Passing palette... | WARNING | Aviso de seaborn. No afecta al resultado.                 |
| ModuleNotFoundError               | ERROR   | Librería no instalada. Ver sección 7.                     |
| FileNotFoundError                 | ERROR   | Archivo no encontrado. Verificar rutas.                   |
| ValueError: DataFrame.dtypes...   | ERROR   | Tipo de dato incompatible. Ver sección 7.                 |
| All the 5 fits failed             | ERROR   | Error en validación cruzada. Revisar tipos de datos.      |
| OK                                | INFO    | Verificación de instalación correcta (sección 2, paso 4). |

### Ejecución exitosa completa

Una ejecución sin errores de ambos scripts produce:

- 3 archivos .pkl en models/ (modelos entrenados)
- 1 archivo scaler.pkl en models/ (escalador)
- 5 archivos .png en images/ (gráficas de evaluación)
- Tabla de métricas impresa en consola
- Análisis de Recall por sexo impreso en consola

Para cualquier duda adicional, consulte el README.md en la raíz del proyecto o el repositorio en GitHub.